

2023-2024 Langage C – Annexe

Partiel du 27/03/2024 — Durée 2h – Chargée de cours Lélia Blin

1 Listing 1

Listing 1: .

```
1 int a = 3;
2 int b = 7;
3 int res = a++ + b++;
4 printf( "%d - %d - %d", a, b, res );
```

2 Listing 2

Listing 2: .

```
1 double result = 1 / 3;
2 printf( "%lf\n", result );
```

3 Exercice 3D

On définit:

Listing 3: .

```
1 struct point3D {
2     float x;
3     float y;
4     float z;
5 };
```

Écrivez un programme en C qui :

- Déclare une variable p1 de type struct point3D et l'initialise avec les coordonnées (1.5, 2.0, -3.7).
- Déclare et initialise un tableau de trois éléments de type struct point3D avec les coordonnées suivantes :
- Pour le premier élément : (0.0, 0.0, 0.0)
- Pour le deuxième élément : (-1.0, 4.5, 2.3)
- Pour le troisième élément : (5.6, -2.8, 1.9)
- Affiche les coordonnées de la variable p1 ainsi que celles de chaque point du tableau.

4 Chaînes et itérations

Dans la fonction donnée et la fonction demandée, on supposera que chaque ligne des textes considérés, se termine par un retour à la ligne, c'est-à-dire par un \ - tenez en compte. Nous supposons déclaré une fonction `void afficher_page(int k, char *s)` (vous n'avez pas à l'écrire), cette fonction affiche à l'écran le texte stocké à l'adresse `s` sur une largeur maximale `k+1` colonnes - les lignes trop longue sont coupées par autant de retours à la ligne nécessaire. Par exemple, si `k` vaut 1 et si le texte est `"abcde\nfg\nh\n"` le texte affiché sera:

```
ab
cd
e
fg
h
```

Noter que les seuls retours à la ligne ajoutés sont après `"ab"` et `"cd"`: tous les autres font parti du texte.

Ecrire une fonction `char *mettre_en_page(int k, char *s)`. Cette fonction doit allouer (par `malloc`) et construire une chaîne dont le texte sera celui stocké à l'adresse `s` avec tous le retours à la ligne ajoutés par la fonction `void afficher_page(int k, char *s)`. Par exemple, si `k` vaut 1 et si le texte est `"abcde\nfg\nh\n"` le texte produit sera `"ab\ncd\ne\nfg\nh\n"`. Le texte stocké à l'adresse `s` ne devra être lu que deux fois en tout: une fois pour déterminer la taille d'allocation, une fois pour la construction du nouveau texte.

5 Que fait le code

Listing 4: .

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     size_t var1;
6     size_t var2;
7     int *var3;
8 } array;
9
10 array *array_init(size_t var1) {
11     array *arr = malloc(sizeof(array));
12     if (arr == NULL) {
13         return NULL;
14     }
15
16     arr->var3 = malloc(var1 * sizeof(int));
17     if (arr->var3 == NULL) {
18         free(arr);
19         return NULL;
20     }
21
22     arr->var1 = var1;
23     arr->var2 = 0;
24
25     return arr;
26 }
27
28 int main() {
29     array *arr = array_init(10);
30     if (arr == NULL) {
31         printf("Echec.\n");
32         return 1;
33     }
34
35     printf("var1 : %zu\n", arr->var1);
36     printf("var2: %zu\n", arr->var2);
37
38     free(arr->var3);
39     free(arr); var3
40
41     return 0;
42 }
```
